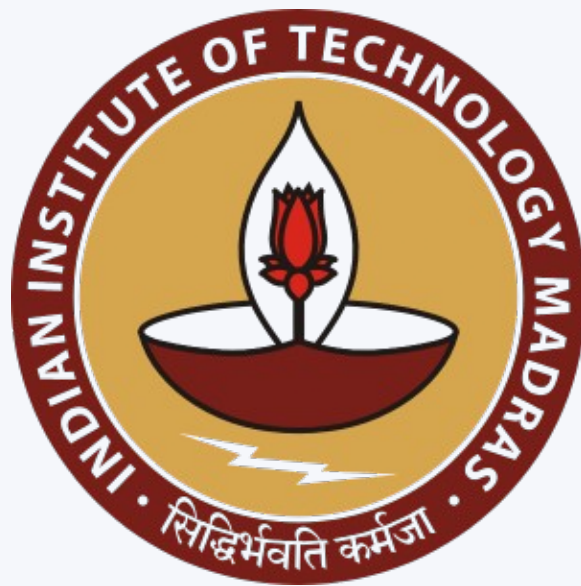


CS3205: INTRODUCTION TO COMPUTER NETWORKS

UNDER

PROF. MUKULIKA MAITY

ASSIGNMENT 4



KARTHIK KASHYAP K

EE23B030

CS3205: Introduction to Computer Networks

ASSIGNMENT 4: DISTANCE VECTOR ROUTING

KARTHIK KASHYAP K

Contents

0	Introduction	2
0.1	Image and Code	2
0.2	Convenience Code	2
1	Question 1: Support Weighted Routing	3
2	Question 2: Implement Poisoned Reverse	4
3	Question 3: Simulate Link Failure	5

§0 Introduction

§0.1 Image and Code

Since the codes and images are best viewed externally than in a pdf file, I have also attached the respective codes and images used in this assignment in this external site: [Assignment 4 Files](#). This also contains a small amount of info that wouldn't fit into this assignment file, but need mentioning (but can otherwise be ignored).

§0.2 Convenience Code

I have written a script to compile and run the codes based on the question number, which can also be found in the site [Assignment 4 Files](#).

§1 Question 1: Support Weighted Routing

Modify the code to support link costs instead of unit hop count.

- (a) Extend the input parser to read link cost.
- (b) Modify data structures to store cost per interface.
- (c) Update routing logic:

$$\text{new_cost} = \text{received_cost} + \text{link_cost}$$

- (d) Ensure the shortest path is computed using the minimum total cost.

Solution

In the original routing implementation, every link was treated as having unit cost. Therefore, the routing table represented the shortest path in terms of hop count only. For Q1, the code was modified so that every interface stores an associated link cost, and route computation is based on the minimum total path cost instead of the minimum number of hops.

The input parser was extended to support a cost field at the end of each topology line:

```
1 Node own_ip neighbor_ip neighbor_node cost
```

If the cost is not specified, the implementation uses a default cost of 1, which keeps the original sample input valid.

In the Data Structure, we add another attribute to store the link cost. This allows the receiving node to determine the cost of the link over which a routing update was received. The distance vector update rule was changed from unit-hop addition to weighted cost addition:

```
1 new_cost = received_cost + link_cost
```

If a destination is not present in the current routing table, a new entry is added. If the destination already exists, the entry is updated only when the newly computed cost is smaller than the existing cost. Routes to the node's own interfaces are not learned through neighbors, since those routes should remain at cost 0.

For the given input along with link costs, this is the output:

```
1 Printing the routing tables after the convergence
2 A:
3 10.0.0.1 | 10.0.0.1 | 10.0.0.1 | 0
4 10.0.0.21 | 10.0.0.21 | 10.0.0.1 | 5
5 10.0.1.23 | 10.0.0.21 | 10.0.0.1 | 5
6 10.0.1.3 | 10.0.0.21 | 10.0.0.1 | 8
7 B:
8 10.0.0.21 | 10.0.0.21 | 10.0.0.21 | 0
9 10.0.1.23 | 10.0.1.23 | 10.0.1.23 | 0
10 10.0.1.3 | 10.0.1.3 | 10.0.1.23 | 3
11 10.0.0.1 | 10.0.0.1 | 10.0.0.21 | 6
12 C:
13 10.0.1.3 | 10.0.1.3 | 10.0.1.3 | 0
14 10.0.0.21 | 10.0.1.23 | 10.0.1.3 | 9
15 10.0.1.23 | 10.0.1.23 | 10.0.1.3 | 9
16 10.0.0.1 | 10.0.1.23 | 10.0.1.3 | 15
```

§2 Question 2: Implement Poisoned Reverse

Extend the DVR implementation to support Poisoned Reverse

(a) Modify `sendMsg()`:

- Routing updates must be neighbor-specific.
- For each neighbor Y:
 - If a route to destination 'D' uses Y as next hop advertise cost as ∞ .
 - Else $\rightarrow$ advertise normal cost.

(b) Define an infinity value (e.g., 16 or 999).

(c) Modify `recvMsg()`:

If a route to D currently uses neighbor Y as next hop $\rightarrow$ update the route using Y's new advertised cost (even if worse).

Solution

Q2 adds Poisoned Reverse to reduce the chance of routing loops in Distance Vector Routing. The main idea is that if a node reaches a destination through a neighbor, then it should not advertise that same destination back to that neighbor as reachable through itself. An infinity value of 999 is used.

The `sendMsg()` function was modified so that the routing update sent to each neighbor is constructed separately. For each neighbor Y, the node checks every routing table entry:

- If the route to destination D uses Y as the next hop, the advertised cost to Y is set to 999.
- Otherwise, the normal route cost is advertised.

The receive-side routing logic was also adjusted. If the current route to a destination already uses the neighbor that sent the update, the route must be updated using the neighbor's new advertised cost even if the cost becomes worse. This is necessary because the neighbor may now be reporting that the destination is unreachable. Therefore, an update is applied when either:

- the new route has a smaller cost, or
- the update came from the neighbor currently used as the next hop.

This behavior is important for correctly handling poisoned routes and avoiding stale next-hop entries.

For the given input, this is the output:

```

1 Printing the routing tables after the convergence
2 A:
3 10.0.0.1 | 10.0.0.1 | 10.0.0.1 | 0
4 10.0.0.21 | 10.0.0.21 | 10.0.0.1 | 5
5 10.0.1.23 | 10.0.0.21 | 10.0.0.1 | 5
6 10.0.1.3 | 10.0.0.21 | 10.0.0.1 | 8
7 B:
8 10.0.0.21 | 10.0.0.21 | 10.0.0.21 | 0
9 10.0.1.23 | 10.0.1.23 | 10.0.1.23 | 0
10 10.0.1.3 | 10.0.1.3 | 10.0.1.23 | 3
11 10.0.0.1 | 10.0.0.1 | 10.0.0.21 | 6
12 C:
13 10.0.1.3 | 10.0.1.3 | 10.0.1.3 | 0
14 10.0.0.21 | 10.0.1.23 | 10.0.1.3 | 9
15 10.0.1.23 | 10.0.1.23 | 10.0.1.3 | 9
16 10.0.0.1 | 10.0.1.23 | 10.0.1.3 | 15
    
```

§3 Question 3: Simulate Link Failure

After initial convergence:

- Remove a link OR set its cost to infinity.
- Continue running the routing algorithm for multiple rounds.
- Print routing tables after reconvergence.

Solution

Q3 studies how the Distance Vector Routing algorithm behaves when a link fails after the network has already converged. The implementation first runs the routing protocol normally, prints the converged routing tables, then simulates a link failure and runs the protocol again until reconvergence.

A bidirectional link failure is simulated by selecting an available link and setting its cost to 999 in both directions. This represents an unreachable link. Any route that was using the failed neighbor as its next hop is also invalidated by setting its cost to 999.

After the link failure, the routing algorithm is run again using the same convergence detection method as Q1 and Q2. Nodes exchange updated distance vectors, including poisoned reverse updates. If an alternate path exists, nodes eventually discover it and update their routing tables. If no alternate path exists, the affected routes remain at cost 999.

The program prints:

- routing tables after initial convergence,
- the failed link,
- routing tables after reconvergence,
- the number of rounds needed to reconverge.

For the provided sample topology, failing the link between A and B disconnects node A from the rest of the network because there is no alternate path from A. As a result, the routes from A to destinations behind B become unreachable and are shown with cost 999. The remaining connected nodes keep valid routes among themselves.

In a topology with an alternate path, the algorithm reconverges by selecting the next lowest-cost available route. The number of rounds required for reconvergence depends on the topology and the order in which routing updates are processed.

Solution (Contd.)

For the given input, this is the output:

```
1  Printing the routing tables after the convergence
2  A:
3  10.0.0.1 | 10.0.0.1 | 10.0.0.1 | 0
4  10.0.0.21 | 10.0.0.21 | 10.0.0.1 | 5
5  10.0.1.23 | 10.0.0.21 | 10.0.0.1 | 5
6  10.0.1.3 | 10.0.0.21 | 10.0.0.1 | 8
7  B:
8  10.0.0.21 | 10.0.0.21 | 10.0.0.21 | 0
9  10.0.1.23 | 10.0.1.23 | 10.0.1.23 | 0
10 10.0.1.3 | 10.0.1.3 | 10.0.1.23 | 3
11 10.0.0.1 | 10.0.0.1 | 10.0.0.21 | 6
12 C:
13 10.0.1.3 | 10.0.1.3 | 10.0.1.3 | 0
14 10.0.0.21 | 10.0.1.23 | 10.0.1.3 | 9
15 10.0.1.23 | 10.0.1.23 | 10.0.1.3 | 9
16 10.0.0.1 | 10.0.1.23 | 10.0.1.3 | 15
17 Initial convergence rounds: 3
18 Simulating link failure: 10.0.0.1 <-> 10.0.0.21
19 Printing the routing tables after link failure reconvergence
20 A:
21 10.0.0.1 | 10.0.0.1 | 10.0.0.1 | 0
22 10.0.0.21 | 10.0.0.21 | 10.0.0.1 | 999
23 10.0.1.23 | 10.0.0.21 | 10.0.0.1 | 999
24 10.0.1.3 | 10.0.0.21 | 10.0.0.1 | 999
25 B:
26 10.0.0.21 | 10.0.0.21 | 10.0.0.21 | 0
27 10.0.1.23 | 10.0.1.23 | 10.0.1.23 | 0
28 10.0.1.3 | 10.0.1.3 | 10.0.1.23 | 3
29 10.0.0.1 | 10.0.0.1 | 10.0.0.21 | 999
30 C:
31 10.0.1.3 | 10.0.1.3 | 10.0.1.3 | 0
32 10.0.0.21 | 10.0.1.23 | 10.0.1.3 | 9
33 10.0.1.23 | 10.0.1.23 | 10.0.1.3 | 9
34 10.0.0.1 | 10.0.1.23 | 10.0.1.3 | 999
35 Rounds to converge after link failure: 2
```